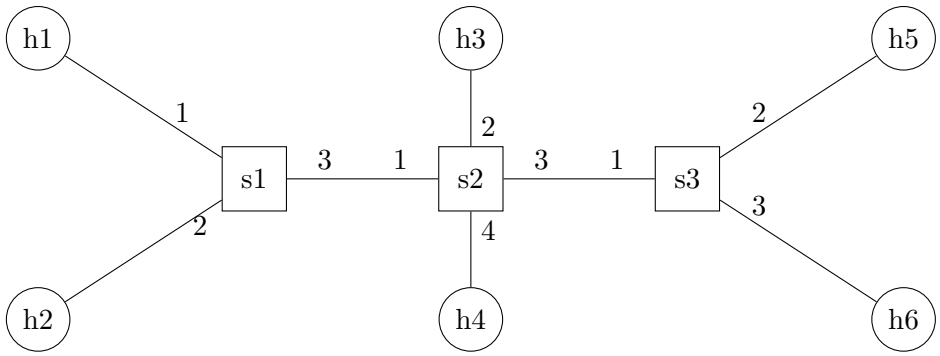
Piattaforme Software per la Rete (Mod. 1) – 13/09/2017

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura.



(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

L'host h1, porta 7843 invia un datagramma UDP a h5, porta 53.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(c) L'host h5 risponde al datagramma UDP precedente. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

L'host h2, porta 1234 apre una connessione TCP verso h6, porta 53.

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(e) L'host h6 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(f) L'host h2 conclude il three-way handshake. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(g) Scrivere il contenuto finale delle tabelle openflow.

Programma 1 (1/2) – Switch Features

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(eth_type = IPv4, ip_proto = UDP, udp_dst = 53)
        actions = [ parser.setField(udp_dst = 5353),
                    parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        match = parser.OFPMatch(eth_type = IPv4, ip_proto = UDP, udp_src = 5353)
        actions = [ parser.setField(udp_src = 53),
                    parser.OFPActionOutput(1) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)
    data = msg.data

    if (datapath.id == 2) and (pkt_tcp is not None):
        match = parser.OFPMatch(
            eth_dst=pkt_eth.src,eth_src=pkt_eth.dst,eth_type=IPv4,ip_proto=TCP)
        actions = [ parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (datapath.id == 1) and (in_port < 3):
        outport = 3
    elif (datapath.id == 3) and (in_port > 1):
        outport = 1
    else:
        outport = ofproto.OFPP_FLOOD
    actions = [parser.OFPActionOutput(outport)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri un sistema di misura del traffico basato su *consistent hashing*. In ogni nodo, ad ogni pacchetto è associato un *flow id* F tale che connessioni TCP distinte abbiano flow id distinti. Se F è già presente in un filtro di Bloom, si incrementa un contatore R. Altrimenti F è aggiunto al filtro. Il filtro è costituito da un array di $m = 1\,024$ bit e $k = 4$ funzioni hash.

Ad uno di questi nodi arrivano pacchetti a velocità $C = 1$ Mbit/s. La lunghezza massima di un pacchetto è $M = 1\,500$ byte. La lunghezza media di una connessione TCP è $L = 1\,000\,000$ byte. Il sistema di misura è resettato ogni $T = 1$ ora.

(a) Calcolare il valore massimo che può assumere il contatore R.

(b) Calcolare la probabilità che il filtro di Bloom restituisca un falso positivo al termine dell'intervallo di osservazione.

Si considerino le seguenti funzioni hash:

$$
\begin{aligned}
g_1(x) &= x \bmod 1024 \\
g_2(x) &= 2g_1(x) \bmod 1024 \\
g_3(x) &= g_1(x) + g_2(x) \bmod 1024 \\
g_4(x) &= 3x + 1 \bmod 1024
\end{aligned}
$$

(c) Spiegare se le funzioni hash definite sopra sono una buona scelta per implementare il filtro di Bloom.

(d) Dire quali sono i bit posti a 1 dopo l'inserimento di $x = 3$.

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Transport Protocol
32.0.0.0/5	*	*	53	UDP
68.0.0.0/6	32.0.0.0/6	*	*	ICMP
*	68.0.0.0/8	*	53	*
80.0.0.0/6	*	*	123	UDP
76.0.0.0/6	80.0.0.0/8	21–22	*	UDP
*	76.0.0.0/8	25	80	*

(a) Definire il bitvector set per ciascun campo.

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie con passo 3. Disegnare il trie.

(c) Si consideri il pacchetto (82.10.0.1, 82.10.0.2, 21, 123, UDP). Elencare le maschere identificate per ogni campo e identificare gli eventuali match.

Esercizio 4

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1: 250, 50, 150

Coda 2: 225, 50, 50

Coda 3: 100, 100, 50

Coda 4: –

Al tempo $t = 400$ ns si presenta alla coda 4 un pacchetto di lunghezza 100. Il valore del quanto è 100 per tutte le code. La lunghezza massima dei pacchetti nel sistema è 250 byte.

(a) Simulare l'algoritmo DRR, indicando il tempo al quale termina ogni trasmissione ed il deficit residuo.

(b) Scegliere i quanti delle varie code in modo da garantire che tutte le code possano trasmettere almeno un pacchetto ad ogni iterazione e che le code 1 e 2 abbiano una banda doppia rispetto alle code 3 e 4. Ripetere la simulazione con i nuovi valori dei quanti.